

Assignment 3 — dbt Integration with BigQuery

Case Study: ShopSphere E-Commerce Analytics Project

ShopSphere has moved its raw application data into BigQuery. Analysts currently join raw tables manually, and each analyst writes slightly different business logic.

The data engineering team wants you to introduce dbt so the BigQuery source data becomes clean, reusable, documented and tested.

Your task is to build a small dbt project demonstrating:

- BigQuery connectivity.
- dbt Sources.
- Staging models.
- Model dependencies.
- `source()` and `ref()`.
- Relationships.
- Data tests.
- Business marts.
- Documentation and lineage.

BigQuery Source Dataset

Use:

`raw_retail`

Create these four source tables with the given records.

Table 1 — customers

customer_id	customer_name	city	customer_segment	signup_date
C001	Ananya	Bengaluru	PREMIUM	2026-06-01
C002	Ravi	Chennai	STANDARD	2026-06-05
C003	Meera	Hyderabad	PREMIUM	2026-06-10
C004	Arun	Pune	STANDARD	2026-06-15
C005	Divya	Bengaluru	STANDARD	2026-06-20

Table 2 — products

product_id	product_name	category	current_price
P101	Laptop	ELECTRONICS	50000
P102	Wireless Mouse	ELECTRONICS	1000
P103	Running Shoes	FASHION	3500
P104	Coffee Maker	HOME	4500
P105	Backpack	FASHION	2200

Table 3 — orders

order_id	customer_id	order_date	order_status	loaded_at
O1001	C001	2026-08-01	COMPLETED	2026-08-01 12:00:00
O1002	C002	2026-08-02	SHIPPED	2026-08-02 12:00:00
O1003	C001	2026-08-03	COMPLETED	2026-08-03 12:00:00
O1004	C003	2026-08-03	CANCELLED	2026-08-03 12:00:00
O1005	C004	2026-08-04	COMPLETED	2026-08-04 12:00:00
O1006	C005	2026-08-05	SHIPPED	2026-08-05 12:00:00

Table 4 — order_items

order_item_id	order_id	product_id	quantity	unit_price
I001	O1001	P101	1	50000
I002	O1001	P102	2	1000
I003	O1002	P103	1	3500
I004	O1003	P104	1	4500
I005	O1003	P105	2	2200
I006	O1004	P101	1	50000

order_item_id	order_id	product_id	quantity	unit_price
I007	O1005	P102	3	1000
I008	O1005	P104	1	4500
I009	O1006	P105	1	2200

Required Relationships

customers.customer_id

|

+---- orders.customer_id

orders.order_id

|

+---- order_items.order_id

products.product_id

|

+---- order_items.product_id

dbt Project Requirements

Create a project structure that separates:

source definitions

staging models

intermediate models

business marts

tests/documentation

Staging Layer

Build staging models for all four source tables.

The staging models must:

- Use dbt Sources instead of hard-coded physical raw-table names.
- Keep required business keys.
- Standardize text values where useful.

- Avoid final business aggregation.

Intermediate Model

Create an order-line detail model combining:

- Orders.
- Customers.
- Products.
- Order items.

Calculate line-level sales amount from quantity and unit price.

Cancelled orders must not contribute to final sales metrics.

Mart 1 — Customer Sales Summary

Create one record per qualifying customer with fields such as:

customer_id

customer_name

city

total_orders

total_items

total_revenue

first_order_date

last_order_date

Document how customers with no qualifying sales are handled.

Mart 2 — Product Sales Performance

Create one record per product with:

product_id

product_name

category

total_quantity_sold

total_revenue

distinct_customers

Cancelled orders must not contribute.

Required Tests

Implement suitable tests for:

- Primary keys.
- Required/non-null keys.
- Accepted order-status values.
- Orders-to-customers relationship.
- Order-items-to-orders relationship.
- Order-items-to-products relationship.

Add descriptions for important models and columns.

Source Requirement

All four raw tables must be declared as dbt Sources.

BigQuery Permission Requirement

Use an identity that can:

- Run BigQuery jobs.
- Read raw_retail.
- Build dbt models in a separate analytics dataset.

Avoid using project Owner for the normal dbt developer if it is not required.

Validation Questions

Demonstrate and explain:

- Which models use source()?
- Which models use ref()?
- What lineage did dbt infer?
- What happens when an order-item points to a missing order?
- What happens when an invalid order status is inserted?
- Why can SQL succeed even when data quality is wrong?

Deliverables

Submit the raw BigQuery tables, dbt project structure, connection evidence, Sources YAML, staging models, intermediate model, both marts, tests/documentation YAML, build/test output, lineage screenshot, and final mart results.

Bonus

Add one of these and explain why it helps:

- Seed.
- Snapshot.